

★ Member-only story

Data Engineering +

Apache Iceberg +

Delta Lake +

Lakehouse Architecture +

Big Data +

Delta Lake vs Iceberg: The Next Big Data Battle

The real decision isn't about features anymore. It's about the future of your data platform.



Yamishift

Following ▾

7 min read · Jul 21, 2026

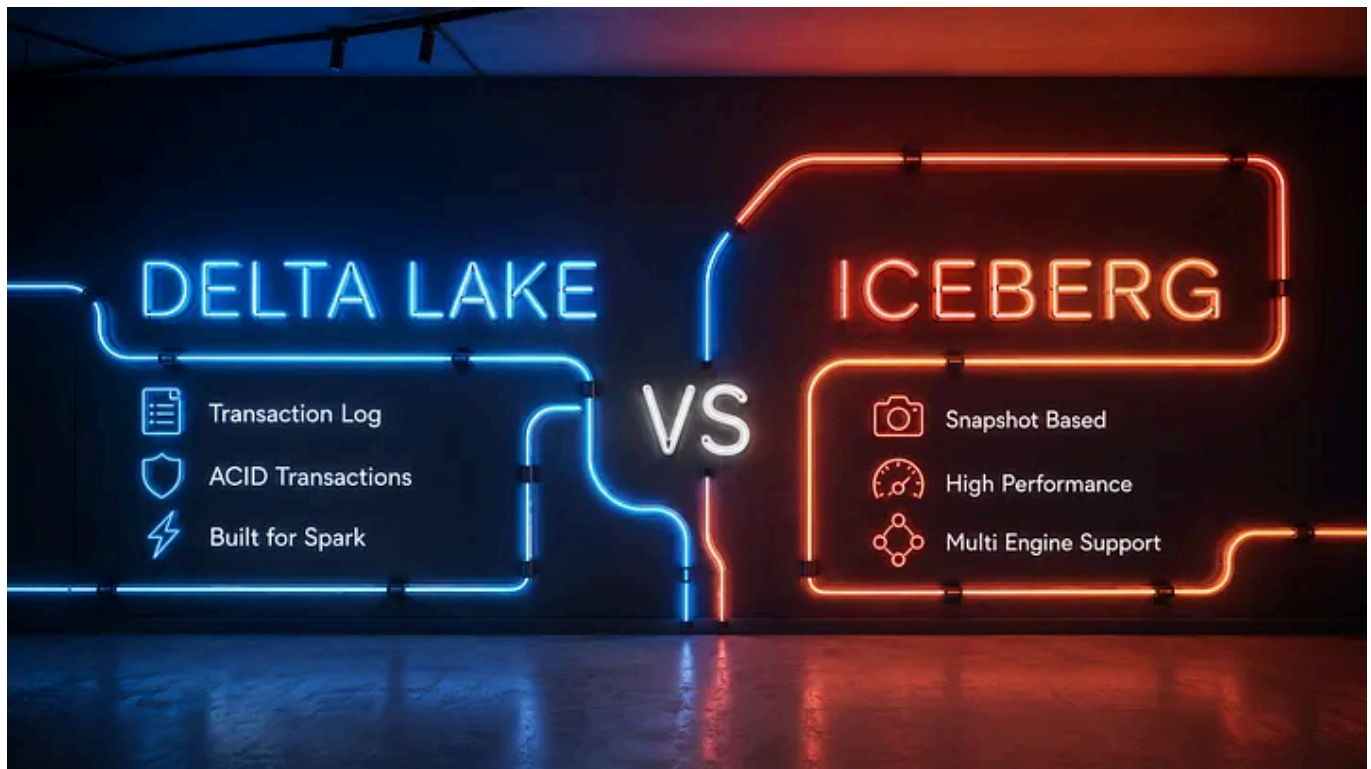


4



2





Choosing between Delta Lake and Apache Iceberg? Learn the real production tradeoffs, architecture differences, performance insights, and engineering lessons.

Everyone Thinks They're Choosing a Table Format.

They're Actually Choosing an Ecosystem.

Around two years ago I joined an architecture review where the agenda looked deceptively simple.

| Should we store our analytical data in Delta Lake or Apache Iceberg?

The meeting was scheduled for thirty minutes.

It lasted almost four hours.

Not because the technologies were complicated.

Because nobody was actually arguing about table formats.

People were arguing about...

- vendor lock-in
- future migrations
- Spark dependency
- governance
- query engines
- cloud costs
- operational ownership

The table format was merely the visible tip of a much larger architectural iceberg.

That's the mistake many teams still make today.

They compare feature checklists.

Production engineers compare **operational consequences**.

One lesson I've learned after building several analytics platforms is this:

Good architectures optimize for change, not for today's benchmarks.

That's exactly why the Delta Lake vs Iceberg discussion has become one of the most important design decisions in modern data engineering. Recent industry trends have also narrowed the feature gap between the two formats, making ecosystem fit more important than ever.

The Old Data Lake Problem

Before understanding why Delta Lake and Iceberg exist, remember what data lakes originally looked like.

```
S3
├── sales/
│   ├── part-001.parquet
│   ├── part-002.parquet
│   └── part-003.parquet
├── users/
│   ├── user1.parquet
│   └── user2.parquet
└── events/
    ├── event1.parquet
    └── event2.parquet
```

Looks simple.

Until reality arrives.

Questions suddenly become difficult.

- Which file is the newest?
- Which files belong to one transaction?
- What if a Spark job fails halfway?
- Can two writers update simultaneously?
- How do you rollback?
- How do you query yesterday's version?

Plain Parquet files have no idea.

Object storage is excellent at storing bytes.

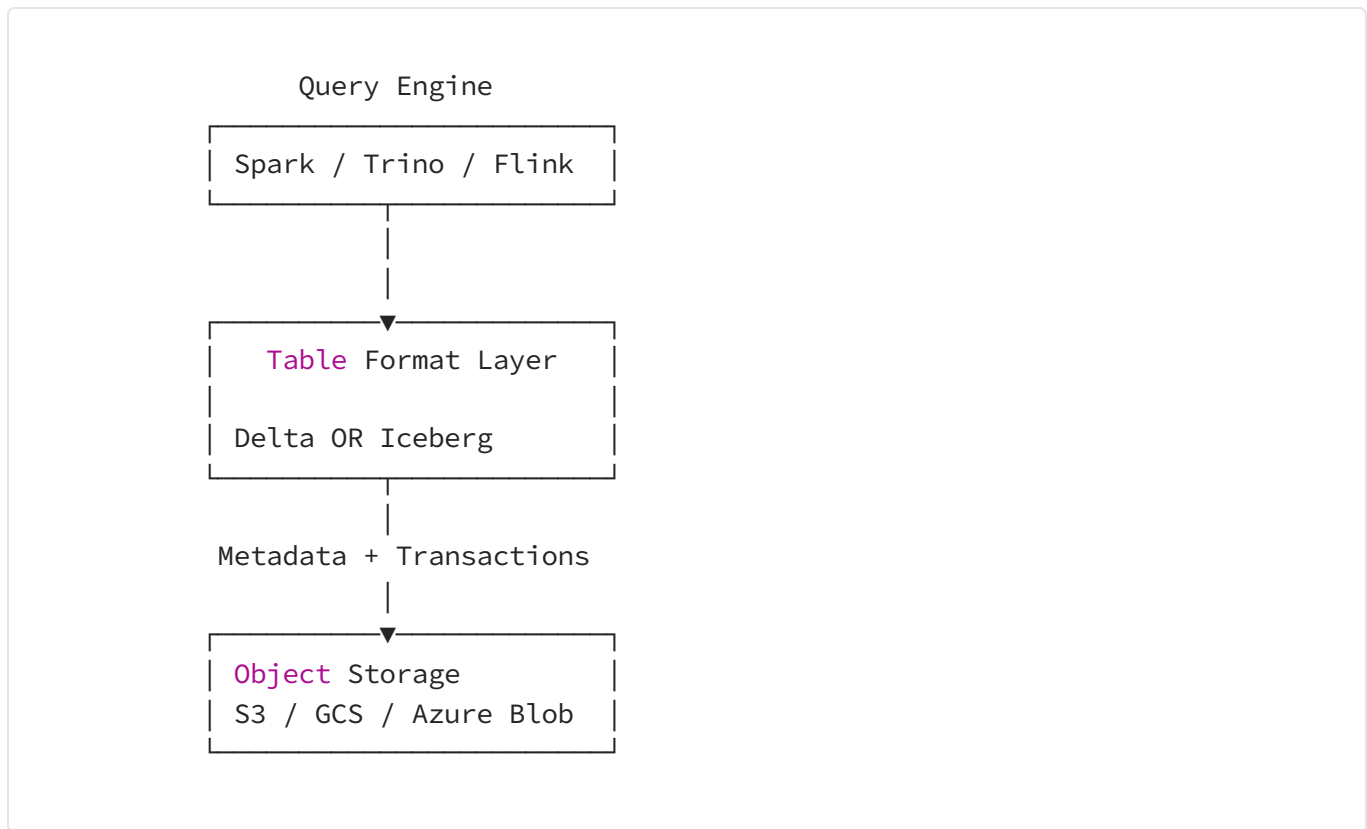
It knows nothing about tables.

That's where **open table formats** changed everything by adding metadata, transaction management, schema evolution, and time travel on top of object storage.

The Real Architecture

Forget marketing diagrams.

Here's what actually exists.



Notice something interesting.

Neither Delta Lake nor Iceberg stores your data differently.

The Parquet files remain Parquet files.

The difference is **how metadata is managed**.

That single architectural decision changes almost everything.

Delta Lake Thinks Like a Database

Delta Lake keeps a transaction log.

Every change becomes another record inside `_delta_log`.

```
sales/  
  
part-001.parquet  
part-002.parquet  
part-003.parquet  
  
_delta_log/  
  
00000001.json  
00000002.json  
00000003.json  
checkpoint.parquet
```

Instead of asking

| “Which files exist?”

Delta asks

| “What transactions happened?”

The latest transaction defines the table.

That sounds simple.

It also makes rollback incredibly elegant.

A simplified transaction might look like this:

```
{
  "commitInfo": {
    "timestamp": 1750602000,
    "operation": "WRITE"
  },
  "add": {
    "path": "part-004.parquet",
    "size": 9234812
  }
}
```

Every write appends another transaction.

Nothing magical.

Just immutable history.

Database engineers tend to love this model because it feels familiar.

Creating a Delta table with PySpark is refreshingly small.

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("analytics")
```



```
.getOrCreate()  
)  
  
df = spark.read.parquet("landing/orders")  
  
(  
    df.write  
        .format("delta")  
        .mode("overwrite")  
        .save("s3://warehouse/orders")  
)
```

Nothing surprising.

That simplicity is one of Delta's biggest strengths.

Now let's update data.

```
from delta.tables import DeltaTable  
  
orders = DeltaTable.forPath(  
    spark,  
    "s3://warehouse/orders"  
)  
  
(  
    orders.alias("target")  
        .merge(  
            updates.alias("source"),  
            "target.id = source.id"  
        )  
        .whenMatchedUpdateAll()  
        .whenNotMatchedInsertAll()  
        .execute()  
)
```

From an application developer's perspective, this feels almost identical to SQL's `MERGE` .

That's intentional.

Delta Lake was designed to make data lakes behave more like databases.

Time travel becomes equally elegant.

```
historic = (  
    spark.read  
        .format("delta")  
        .option("versionAsOf", 42)  
        .load("s3://warehouse/orders")  
)
```

One line.

Instant rollback.

Anyone who has accidentally deployed a broken ETL job at 2 AM knows why this feature is worth its weight in gold.

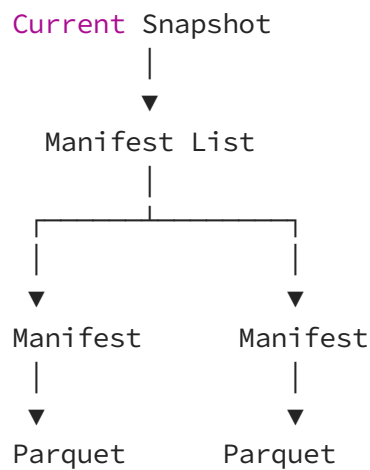
Iceberg Thinks Like Git

Iceberg solves exactly the same problems.

But its mental model is completely different.

Instead of transaction logs...

Iceberg creates **snapshots**.



Rather than replaying transactions...

Iceberg jumps directly to the latest snapshot.

That's a subtle architectural difference.

It becomes a massive advantage once datasets contain millions of files because metadata is organized into manifests and snapshot trees rather than replayed from an append-only log.

Creating an Iceberg table with Spark feels equally straightforward.

```
spark.sql("""  
CREATE TABLE warehouse.sales (  
  
    order_id BIGINT,  
    customer_id BIGINT,  
    total DECIMAL(10,2),  
    created_at TIMESTAMP  
  
)  
USING iceberg  
PARTITIONED BY (days(created_at))  
""")
```

Notice something interesting.

```
PARTITIONED BY (days(created_at))
```

That isn't traditional Hive partitioning.

Iceberg introduces **hidden partitioning**, allowing engines to evolve partition layouts without exposing partition columns to users. This is one of its defining capabilities for long-lived analytical datasets.

In production, writes are usually performed with DataFrame APIs.

```
(  
    df.writeTo("warehouse.sales")  
        .append()  
)
```

Small API.

Huge implications.

The engine updates metadata.

Creates a new snapshot.

Atomically switches the table pointer.

Readers never see partial writes.

One of my favorite Iceberg features appears only after your dataset reaches tens of terabytes.

Suppose today's partitioning strategy looks like this.

```
year  
month
```

day

Three years later...

Business requirements change.

Now your largest customers query by customer region instead of calendar date.

Traditional Hive tables would make this migration painful.

Iceberg lets the table evolve without forcing a rewrite of every historical file.

That's the kind of feature nobody appreciates...

until they're asked to migrate 900 TB over a weekend.

The First Production Mistake Everyone Makes

Most teams spend weeks comparing benchmark charts.

Almost nobody spends one hour asking a much better question:

Who is going to read this data?

That question alone eliminates half the debate.

Imagine three different organizations.

Company A

Spark only
Databricks
Streaming ETL

Company B

Spark
Trino
Flink
Snowflake
DuckDB

Company C

AWS Glue
Athena
EMR
Occasional Spark

All three have identical datasets.

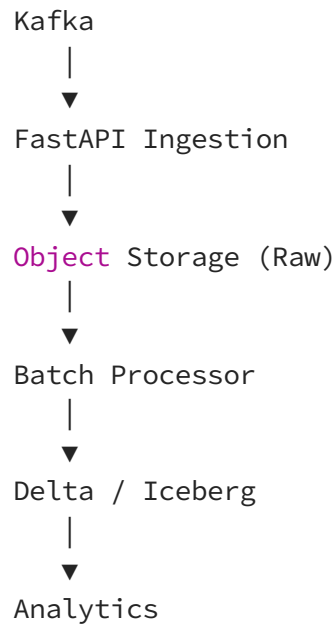
None should necessarily choose the same table format.

Because table formats aren't isolated technologies.

They're infrastructure decisions.

Let's Build a Production Data Pipeline

Suppose we're ingesting clickstream events.



Our ingestion service is intentionally boring.

Reliable software usually is.

```
from fastapi import FastAPI
from pydantic import BaseModel
import uuid

app = FastAPI()

class Event(BaseModel):
    user_id: str
    page: str
    timestamp: str

@app.post("/events")
async def ingest(event: Event):

    event_id = str(uuid.uuid4())

    return {
        "event_id": event_id,
```



```
    "status": "accepted"  
  }
```

Notice something.

The API doesn't care whether downstream storage uses Delta or Iceberg.

That's good architecture.

Storage decisions should stay behind service boundaries.

Next comes the batch job.

```
from pyspark.sql import SparkSession  
  
spark = (  
    SparkSession.builder  
        .appName("clickstream")  
        .getOrCreate()  
)  
  
events = spark.read.json(  
    "s3://landing/events"  
)
```

Now the decision appears.

Delta.

```
(  
  events.write  
    .format("delta")  
    .mode("append")  
    .save("s3://warehouse/events")  
)
```

Or Iceberg.

```
(  
  events.writeTo("warehouse.events")  
    .append()  
)
```

Nearly identical.

Which is why API comparisons are misleading.

The differences emerge months later.

Metadata Is Where Performance Lives

People love benchmarking Parquet.

The smarter engineers benchmark metadata.

Imagine one billion files.

Finding the correct ones matters more than reading them.

Delta performs planning using its transaction log and periodic checkpoints.

```
_delta_log/  
  
000001.json  
000002.json  
000003.json  
...  
  
checkpoint.parquet
```

Readers load the latest checkpoint.

Replay newer commits.

Done.

Iceberg instead builds a metadata tree.

```
Snapshot  
↓  
Metadata File  
↓  
Manifest List  
↓
```

Manifest

↓

Parquet Files

Planning becomes hierarchical.

Entire groups of files disappear before scanning even begins.

That's one reason Iceberg has become popular for very large multi-engine deployments.

Bad Partitioning

Every engineer eventually writes something like this.

```
df.write.partitionBy(  
    "year",  
    "month",  
    "day"  
)
```

Looks reasonable.

Six months later...

Marketing starts querying by customer region.

Oops.

Now your partitions optimize yesterday's workload.

Not today's.

Iceberg's hidden partitioning changes the game.

```
CREATE TABLE sales (  
  id BIGINT,  
  region STRING,  
  created_at TIMESTAMP  
  
)  
  
USING iceberg  
  
PARTITIONED BY (  
  bucket(32, region),  
  days(created_at)  
  
);
```

Applications don't need to know how data is partitioned.

The metadata layer handles it.

Years later you can evolve the partition strategy without rewriting historical data. That's one of Iceberg's biggest operational advantages.

Schema Evolution

Real schemas don't stay still.

Version one.

```
class Order(BaseModel):  
  
    id: int  
    total: float
```

Two months later.

```
class Order(BaseModel):  
  
    id: int  
    total: float  
    currency: str
```

Then...

```
class Order(BaseModel):  
  
    id: int  
    total: float
```

```
currency: str  
discount: float
```

Then finance asks for VAT.

Then logistics wants warehouse IDs.

Then compliance introduces data retention rules.

Schema evolution stops being a feature.

It becomes survival.

Both Delta Lake and Iceberg support schema evolution, but Iceberg's field-ID based metadata generally offers more flexibility for long-lived schemas across multiple engines.

Concurrency Is Where Production Gets Interesting

Imagine three writers.

Open in app ↗



Medium



Search



Write



Writer B

Writer C

All updating the same table.

Naive systems eventually corrupt data.

Modern table formats don't.

Instead they rely on optimistic concurrency.

Simplified logic looks like this.

```
while True:

    snapshot = load_snapshot()

    changes = prepare_changes()

    if commit(snapshot, changes):
        break
```

If another writer wins first...

Retry.

No corrupted metadata.

No half-written tables.

Production engineering is often less about preventing failure...

and more about making failure recoverable.

Compaction Never Stops

Tiny files quietly destroy analytics clusters.

One thousand 5 MB files are almost always slower than five 1 GB files.

Delta users often schedule optimization jobs.

```
OPTIMIZE sales;
```

Many teams also organize files for common filter columns.

```
OPTIMIZE sales  
ZORDER BY (customer_id);
```

Iceberg approaches the same problem through rewrite and compaction procedures.

The objective is identical.

Fewer files.

Less metadata.

Faster planning.

Different implementation.

Same engineering goal.

Time Travel Is More Than Debugging

Many developers think time travel exists for accidental deletes.

That's true.

But the real superpower is reproducibility.

Suppose Finance reports revenue.

Three weeks later someone reruns the query.

Without versioned tables...

Numbers might change.

With versioned tables...

```
historic = (  
    spark.read  
    .format("delta")  
    .option("versionAsOf", 128)
```

```
.load(path)  
)
```

Or in Iceberg.

```
SELECT *  
  
FROM sales  
  
VERSION AS OF 128;
```

Historical analytics become deterministic.

That matters far more than rollback.

Streaming Changes the Equation

If your organization lives inside the Spark ecosystem and continuously ingests streaming data, Delta Lake remains a compelling choice because of its mature integration with Spark and Structured Streaming. ([PuppyGraph](#))

A simplified streaming pipeline might look like this.

```
stream = (  
    spark.readStream  
        .format("kafka")  
        .load()  
)
```

```
(  
    stream.writeStream  
        .format("delta")  
        .option(  
            "checkpointLocation",  
            "/checkpoints/events"  
        )  
        .start()  
)
```

Everything feels cohesive.

That's not an accident.

Delta grew up alongside Spark.

Meanwhile, organizations that routinely query the same datasets from Spark, Trino, Flink, DuckDB, Snowflake, and cloud-native engines often gravitate toward Iceberg because interoperability becomes more valuable than engine-specific optimizations.

One lesson worth remembering:

The best storage format is the one your future architecture won't outgrow.

Big Data +



Written by Yamishift

687 followers · 88 following

Following ▾

minor character writing about code, ai & side projects. build logs, experiments, and late-night notes from learning in public.

